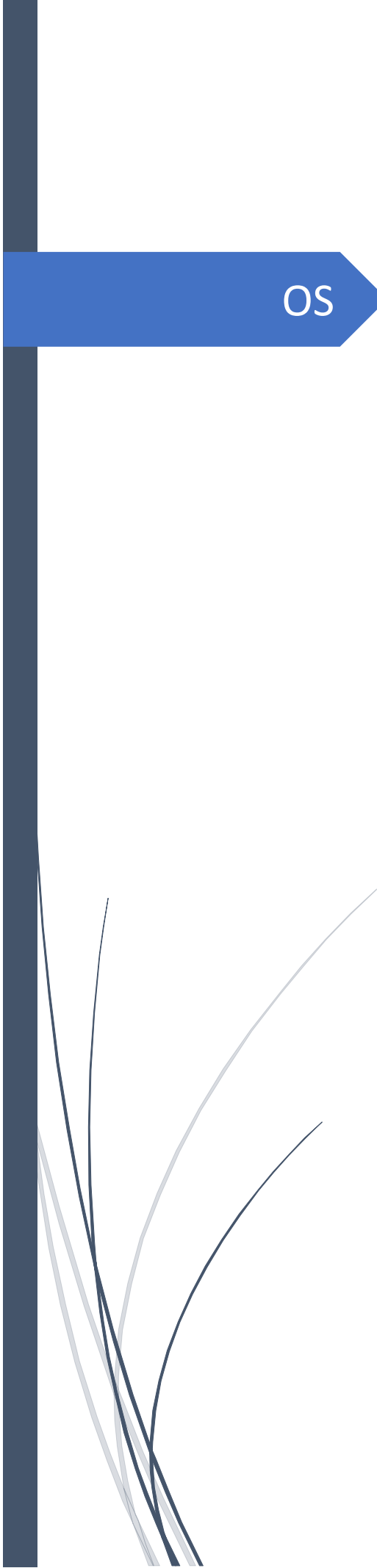




os

Operating System Lab

Er. Sital Prasad Mandal
Info.sitalmandal@gmail.com
www.spm.com.np
<https://github.com/1sp11cs070/OS-CSIT-LAB>

Table of Contents

1. FIFO Page Replacement	3
2. Banker's Algorithm.....	4
3. C-LOOK Disk Scheduling	7
4. C-SCAN Disk Scheduling	9
5. FCFS Scheduling	11
6. LFU Page Replacement	12
7. LOOK Disk Scheduling	13
8. LRU Page Replacement	15
9. Optimal Page Replacement.....	17
10. Priority Scheduling	19
11. Producer-Consumer Problem	21
12. Readers-Writers Problem	22
13. SCAN Disk Scheduling.....	24
14. Semaphore Demonstration.....	26
15. SJF Scheduling	27
16. SSTF Disk Scheduling.....	28
Operating System Viva Questions with Answers.....	30
1. What is an Operating System (OS)?	30
2. What are the main functions of an Operating System?	30
3. What is a Process?	30
4. What is a Thread?	30
5. What is CPU Scheduling?	30
6. What is FCFS Scheduling?	30
7. What is SJF Scheduling?	30
8. What is Priority Scheduling?	30
9. What is Round Robin Scheduling?	31
10. What is Burst Time?	31
11. What is Waiting Time?	31
12. What is Turnaround Time?	31
13. What is Memory Management?	31
14. What is Virtual Memory?	31
15. What is Paging?	31
16. What is a Page Fault?	31
17. What is Page Replacement?	31

18. What is FIFO Page Replacement?	31
19. What is LRU Page Replacement?	32
20. What is Optimal Page Replacement?.....	32
21. Which page replacement algorithm gives minimum page faults?	32
22. What is Thrashing?.....	32
23. What is Deadlock?.....	32
24. What are the four necessary conditions for Deadlock?	32
25. What is Banker's Algorithm?.....	32
26. What is Safe State?	32
27. What is Semaphore?	32
28. What is Mutex?	33
29. What is the Producer-Consumer Problem?	33
30. What is the Readers-Writers Problem?	33
31. What is File Management?	33
32. What is a File?	33
33. What is Disk Scheduling?	33
34. Name Disk Scheduling Algorithms.	33
35. What is SSTF?	33
36. What is SCAN Scheduling?	33
37. What is C-SCAN?	34
38. What is LOOK Scheduling?	34
39. What is C-LOOK Scheduling?.....	34
40. Which disk scheduling algorithm gives the best average seek time?.....	34

1. FIFO Page Replacement

Question:

Write a C program to implement the FIFO (First In First Out) Page Replacement algorithm and display the total number of page faults.

```
#include<stdio.h>

int main() {
    int page[20], frame[10], pfcount=0, np, nf;
    int i, j, k=0, available;
    float pfrate;

    printf("Enter no. of frames: \n");
    scanf("%d", &nf);
    printf("Enter no. of pages:\n");
    scanf("%d", &np);
    printf("Enter reference string:\n");
    for(i=0; i<np; i++) {
        scanf("%d", &page[i]);
    }

    //putting -1(blank) initially in all frames
    for(i=0; i<nf; i++)
        frame[i]=-1;

    //printing headings
    printf("Ref String\tFrames\n");
    for(i=0; i<np; i++) {
        //printing all pages
        printf("%d\t\t\t", page[i]);

        available=0;
        //scanning frames
        for(j=0; j<nf; j++) {
            //checking page is in frame or not
            if(page[i]==frame[j]) {
                available=1;
                printf("No Replacement\n");
                break;
            }

            if(available==0) { //not available in frame, page fault
                //Replacing page now
                frame[k]=page[i];
                k=(k+1)%nf; //calculate frame no. for next replacement
                pfcount++;
                //printing frame values
                for(j=0; j<nf; j++)
                    printf("%d\t", frame[j]);
                printf("\n");
            }
        }
    }
    printf("\nTotal Page Fault=%d", pfcount);
    pfrate=(float)pfcount/(float)np;
    printf("\nPage Fault(Miss) Ratio=%.2f", pfrate*100);
    return 0;
}
```

Sample Input

```
Enter no. of frames:
3
Enter no. of pages:
12
Enter reference string:
1 2 3 4 1 2 5 1 2 3 4 5
```

Sample Output

Ref String	Frames
1	1 -1 -1
2	1 2 -1
3	1 2 3
4	4 2 3
1	4 1 3
2	4 1 2
5	5 1 2
1	No Replacement
2	No Replacement
3	5 3 2
4	5 3 4
5	No Replacement

```
Total Page Fault = 9
Page Fault (Miss) Ratio = 75.00
```

2. Banker's Algorithm

Question:

Write a C program to implement Banker's Algorithm and determine whether the system is in a safe state.

```
#include<stdio.h>

int main()
{
    int allocation[10][5],max[10][5],need[10][5],available[10],finish[10],sq[10];
    //res //pro
    int n,m,i,j,k,count,count1=0;

    printf("Enter the no of processes : ");
    scanf("%d", &n);

    printf("\nEnter the no of resource type : ");
    scanf("%d", &m);

    printf("\n\nEnter the Max Matrix for each process : ");
    for(i = 0; i < n; i++)
    {
        printf("\nFor process %d : \n", i+1);
        for(j = 0; j < m; j++)
            scanf("%d", &max[i][j]);
    }

    printf("\n\nEnter the allocation for each process : ");
```

```

for(i = 0; i < n; i++)
{
    printf("\nFor process %d : \n",i+1);
    for(j = 0; j < m; j++)
        scanf("%d", &allocation[i][j]);
}

printf("\nEnter the Available Resources : \n");
for(j = 0; j < m; j++)
    scanf("%d", &available[j]);

printf("\n The Need Matrix is as Follows : \n");
for(i=0;i<n;i++)
{
    for(j=0;j<m;j++)
    {
        need[i][j]= max[i][j]-allocation[i][j];
        printf("\t %d",need[i][j]);
    }
    printf("\n");
}

//checking safe state
for(i=0;i<10;i++)
    finish[i]=0; //setting finish of all process to 0

do{
    for(k=0;k<n;k++) //outer loop
    {
        for(i=0;i<n;i++) //loop for process
        {
            if(finish[i]==0) //checking process is finished or not.
            {
                count=0; //setting 1st counter as 0 initially.
                for(j=0;j<m;j++) //loop for resource
                {
                    if(available[j]>=need[i][j]) //comparing available with need
matrix.

                    count++; //incrementing 1st counter.
                }

                if(count==m) //if counter equals no of resources
                    //must be available all resource type..
                    //there should be all or none basis..
                {
                    count1++; //incrementing second counter
                    finish[i]=1; //setting process as finished
                    sq[count1-1]=i+1; //putting process in safe sequence array
                    for(j=0;j<m;j++)
                    {
                        available[j]=available[j]+allocation[i][j]; //incrementing
available

                    }
                    break;
                }
            }
        }
    }

    if(count1!=n) //if second counter not equals no. of process, then unsafe
state
    {
        printf("\n-----IT'S AN UNSAFE STATE-----");
        break;
    }
}while(count1!=n);

```

```

if(count1==n)    //if second counter not equals no. of process, then safe state
{
    printf("\n *****IT'S A SAFE STATE*****");
    printf("\n The Safe Sequence is...\n");
    for(i=0;i<n;i++)
        printf("\t P%d",sq[i]);
    printf("\n");
    printf("\n The available array is now : ");
    for(j=0;j<m;j++)
        printf("\t %d",available[j]);
}

return 0;
}

```

Sample Input

Enter the no. of processes: 5
 Enter the no. of resource types: 3

Max Matrix

```

7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

```

Allocation Matrix

```

0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

```

Available:

```

3 3 2

```

Sample Output

The Need Matrix is:

```

7 4 3
1 2 2
6 0 0
0 1 1
4 3 1

```

IT'S A SAFE STATE

Safe Sequence:

```

P2 P4 P5 P1 P3

```

3. C-LOOK Disk Scheduling

Question:

Write a C program to implement the C-LOOK Disk Scheduling algorithm.

```
#include<stdio.h>
#include<stdlib.h>

void printseq(int value){
    printf("%d->",value);
}

int main()
{
    int RQ[100],i,j,n,TotalHeadMoment=0,initial,size,move;
    printf("Enter the number of Requests\n");
    scanf("%d",&n);
    printf("Enter the Requests sequence\n");
    for(i=0;i<n;i++){
        scanf("%d",&RQ[i]);
    }
    printf("Enter initial head position\n");
    scanf("%d",&initial);
    printf("Enter total disk size\n");
    scanf("%d",&size);
    printf("Enter the head movement direction for high 1 and for low 0\n");
    scanf("%d",&move);

    // logic for C-look disk scheduling

    /*logic for sort the request array */
    for(i=0;i<n;i++){
        for( j=0;j<n-i-1;j++){
            if(RQ[j]>RQ[j+1])
            {
                int temp;
                temp=RQ[j];
                RQ[j]=RQ[j+1];
                RQ[j+1]=temp;
            }
        }
    }

    int index;
    for(i=0;i<n;i++){
        if(initial<RQ[i])
        {
            index=i;
            break;
        }
    }

    // if movement is towards high value
    if(move==1)
    {
        for(i=index;i<n;i++){
            TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
            initial=RQ[i];
            printseq(initial);
        }
    }
}
```

```

    }

    for( i=0;i<index;i++)
    {
        TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
        initial=RQ[i];
        printseq(initial);
    }
}
// if movement is towards low value
else
{
    for(i=index-1;i>=0;i--)
    {
        TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
        initial=RQ[i];
        printseq(initial);
    }

    for(i=n-1;i>=index;i--)
    {
        TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
        initial=RQ[i];
        printseq(initial);
    }
}

printf("\nTotal head movement is %d",TotalHeadMoment);
return 0;
}

```

Sample Input

```

Enter number of requests:
8
Request sequence:
98 183 37 122 14 124 65 67
Initial head position:
53
Disk size:
200
Direction:
1

```

Sample Output

```

65->67->98->122->124->183->14->37->
Total head movement is 322

```

4. C-SCAN Disk Scheduling

Question:

Write a C program to implement the C-SCAN Disk Scheduling algorithm.

```
#include<stdio.h>
#include<stdlib.h>

void printseq(int value){
    printf("%d->",value);
}

int main()
{
    int RQ[100],i,j,n,TotalHeadMoment=0,initial,size,move;
    printf("Enter the number of Requests\n");
    scanf("%d",&n);
    printf("Enter the Requests sequence\n");
    for(i=0;i<n;i++){
        scanf("%d",&RQ[i]);
    }
    printf("Enter initial head position\n");
    scanf("%d",&initial);
    printf("Enter total disk size\n");
    scanf("%d",&size);
    printf("Enter the head movement direction for high 1 and for low 0\n");
    scanf("%d",&move);

    // logic for C-Scan disk scheduling

    /*logic for sort the request array */
    for(i=0;i<n;i++){
        for( j=0;j<n-i-1;j++){
            if(RQ[j]>RQ[j+1])
            {
                int temp;
                temp=RQ[j];
                RQ[j]=RQ[j+1];
                RQ[j+1]=temp;
            }
        }
    }

    int index;
    for(i=0;i<n;i++){
        if(initial<RQ[i])
        {
            index=i;
            break;
        }
    }

    // if movement is towards high value
    if(move==1)
    {
        for(i=index;i<n;i++){
            TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
            initial=RQ[i];
            printseq(initial);
        }
    }
}
```

```

    }
    // last movement for max size
    TotalHeadMoment=TotalHeadMoment+abs(size-RQ[i-1]-1);
    /*movement max to min disk */
    TotalHeadMoment=TotalHeadMoment+abs(size-1-0);
    initial=0;
    for( i=0;i<index;i++)
    {
        TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
        initial=RQ[i];
        printseq(initial);
    }
}
// if movement is towards low value
else
{
    for(i=index-1;i>=0;i--)
    {
        TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
        initial=RQ[i];
        printseq(initial);
    }
    // last movement for min size
    TotalHeadMoment=TotalHeadMoment+abs(RQ[i+1]-0);
    /*movement min to max disk */
    TotalHeadMoment=TotalHeadMoment+abs(size-1-0);
    initial =size-1;
    for(i=n-1;i>=index;i--)
    {
        TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
        initial=RQ[i];
        printseq(initial);
    }
}

printf("\nTotal head movement is %d",TotalHeadMoment);
return 0;
}

```

Sample Input

Requests: 98 183 37 122 14 124 65 67
 Initial Head: 53
 Disk Size: 200
 Direction: 1

Sample Output

65->67->98->122->124->183->14->37->
 Total head movement is 382

5. FCFS Scheduling

Question:

Write a C program to implement the FCFS CPU Scheduling algorithm.

```
//FCFS Scheduling Algorithm
#include <stdio.h>

int main() {
    //variables declaration
    //input - process no, BT
    //processing - WT, TAT, AWT, ATAT
    int p[10],bt[10],n,i,j,wt[10],tat[10];
    float awt=0,atat=0;

    printf("Enter no. of process (Max 10)");
    scanf("%d",&n);

    //entering process no. and burst time
    printf("\nEnter and burst time");
    for(i=0;i<n;i++){
        p[i]=i+1;    // p[0]= 1   p[1]=2   p[2]=3
        //scanf("%d",&p[i]);
        scanf("%d",&bt[i]);
    }

    //processing and calculating WT,TAT
    //calculating waiting time
    wt[0]=0; //wating time of first process is always 0
    for(i=1;i<n;i++){    //for calculating wt of process
        //we need to add bt of all previous processes
        //scan all previous process
        wt[i]=0;    //set 0 initally
        for(j=0;j<i;j++){    //for scanning and getting bt of previous processes.
            wt[i]=wt[i]+bt[j];    //wt calculated
        }

        //calculating average
        awt=awt+wt[i];
    }

    //calculating TAT
    for(i=0;i<n;i++){
        tat[i]=wt[i]+bt[i];
        //calculating average
        atat=atat+tat[i];
    }

    //displaying data
    printf("\nProcess\tBT\tWT\tTAT\n");
    for(i=0;i<n;i++)
        printf("P%d\t%d\t%d\t%d\n",p[i],bt[i],wt[i],tat[i]);

    //diplaying Average
    printf("\nAWT=%.2f",awt/n);
    printf("\nATAT=%.2f",atat/n);

    return 0;
}
```

Sample Input

```
Enter number of processes:
3
Burst Time:
24
3
3
```

Sample Output

Process	BT	WT	TAT
P1	24	0	24
P2	3	24	27
P3	3	27	30

```
Average Waiting Time = 17.00
Average Turnaround Time = 27.00
```

6. LFU Page Replacement

Question:

Write a C program to implement the LFU (Least Frequently Used) Page Replacement algorithm.

```
#include<stdio.h>
int main()
{
    int page[20], i, j, k, np, nf, count[10], frame[10], min, pfcunt=0;
    printf("\nEnter the available no. of frames -- ");
    scanf("%d", &nf);

    printf("\nEnter number of page references -- ");
    scanf("%d", &np);

    printf("\nEnter the reference string -- ");
    for(i=0; i<np; i++)
        scanf("%d", &page[i]);

    for(i=0; i<nf; i++)
    {
        count[i]=0; //setting count as 0 initially
        frame[i]=-1; //setting frames empty
    }

    //scanning all pages
    for(i=0; i<np; i++)
    {
        //scanning all frames
        for(j=0; j<nf; j++)
            //checking page available in frame or not
            if(page[i]==frame[j]) //available
            {
                count[j]++; //just increase the count
            }
    }
}
```

```

        break;
    }

    //logic for page replacement
    if(j==nf) //if not available
    {
        min = 0;
        for(k=1;k<nf;k++)
            if(count[k]<count[min])
                min=k;

        frame[min]=page[i];
        count[min]=1;
        pfcount++;
    }

    printf("\n");
    for(j=0;j<nf;j++)
        printf("%d\t",frame[j]);
}

printf("\n\n Total number of page faults -- %d",pfcount);
return 0;
}

```

Sample Input

Frames: 3
 Pages: 7
 Reference String:
 1 2 3 1 4 5 2

Sample Output

Total Page Faults = 6

7. LOOK Disk Scheduling

Question:

Write a C program to implement the LOOK Disk Scheduling algorithm.

```

#include<stdio.h>
#include<stdlib.h>

void printseq(int value){
    printf("%d->",value);
}

int main()

```

```

{
    int RQ[100],i,j,n,TotalHeadMoment=0,initial,size,move;
    printf("Enter the number of Requests\n");
    scanf("%d",&n);
    printf("Enter the Requests sequence\n");
    for(i=0;i<n;i++)
        scanf("%d",&RQ[i]);
    printf("Enter initial head position\n");
    scanf("%d",&initial);
    printf("Enter total disk size\n");
    scanf("%d",&size);
    printf("Enter the head movement direction for high 1 and for low 0\n");
    scanf("%d",&move);

    // logic for look disk scheduling

    /*logic for sort the request array */
    for(i=0;i<n;i++)
    {
        for(j=0;j<n-i-1;j++)
        {
            if(RQ[j]>RQ[j+1])
            {
                int temp;
                temp=RQ[j];
                RQ[j]=RQ[j+1];
                RQ[j+1]=temp;
            }
        }
    }

    int index;
    for(i=0;i<n;i++)
    {
        if(initial<RQ[i])
        {
            index=i;
            break;
        }
    }

    // if movement is towards high value
    if(move==1)
    {
        for(i=index;i<n;i++)
        {
            TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
            initial=RQ[i];
            printseq(initial);
        }

        for(i=index-1;i>=0;i--)
        {
            TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
            initial=RQ[i];
            printseq(initial);
        }
    }
    // if movement is towards low value
    else
    {
        for(i=index-1;i>=0;i--)
        {
            TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
            initial=RQ[i];
            printseq(initial);
        }
    }
}

```

```

        for(i=index;i<n;i++)
        {
            TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
            initial=RQ[i];
            printseq(initial);

        }
    }

    printf("\nTotal head movement is %d",TotalHeadMoment);
    return 0;
}

```

Sample Input

Requests:
 98 183 37 122 14 124 65 67
 Initial Head:
 53
 Direction:
 1

Sample Output

65->67->98->122->124->183->37->14->
 Total head movement is 299

8. LRU Page Replacement

Question:

Write a C program to implement the LRU (Least Recently Used) Page Replacement algorithm.

```

#include<stdio.h>

//finding least recently used in past...
int findLRU(int duration[],int n){
    int i,pos=0,min=duration[0];
    for(i=1;i<n;i++){
        if(duration[i]<min){
            min=duration[i];
            pos=i;
        }
    }
    return pos;
}

int main(){
    int page[20],frame[10],pfcount=0,np,nf;

```

```

int i,j,flag1,flag2;
float pfrate;
int count=0,duration[20],pos;

printf("Enter no. of frames: \n");
scanf("%d",&nf);
printf("Enter no. of pages:\n");
scanf("%d",&np);
printf("Enter reference string:\n");
for(i=0;i<np;i++){
    scanf("%d",&page[i]);
}

//putting -1(blank) initially in all frames
for(i=0;i<nf;i++)
    frame[i]=-1;

//printing headings
printf("Ref String\tFrames\n");
for(i=0;i<np;i++){
    //printring all pages
    printf("%d\t\t\t",page[i]);

    /*setting both flag initially as 0
    flag1 - used for checking frame empty or not
    flag2 - used for checking frame empty and page available or not
    */
    flag1=flag2=0;

    //scanning all frames
    for(j=0;j<nf;j++){
        //checking page is on the frame or not
        if(frame[j]==page[i]){ //available
            //No page fault
            count++;
            duration[j]=count;
            flag1=flag2=1;
            break;
        }

        //checking frame empty or not
        if(flag1==0 && frame[j]==-1){ //frame is empty
            count++;
            pfcount++;
            frame[j]=page[i];
            duration[j]=count;
            flag2=1;
            break;
        }
    } //end of j loop

    //replacing the page
    if(flag2==0){
        pos=findLRU(duration, nf);
        count++;
        pfcount++;
        frame[pos]=page[i];
        duration[pos]=count;
    }

    //printing frame values
    for(j=0;j<nf;j++)
        printf("%d\t",frame[j]);
    printf("\n");
}

printf("\nTotal Page Fault=%d",pfcount);
pfrate=(float)pfcount/(float)np;

```

```

printf("\nPage Fault(Miss) Ratio=%.2f",pfrate*100);
return 0;
}

```

Sample Input

```

Frames: 3
Pages: 12
Reference String:
1 2 3 4 1 2 5 1 2 3 4 5

```

Sample Output

```

Total Page Fault = 10
Page Fault (Miss) Ratio = 83.33%

```

9. Optimal Page Replacement

Question:

Write a C program to implement the Optimal Page Replacement algorithm.

```

#include<stdio.h>
int main()
{
    int nf, np, frame[10], page[30], temp[10];
    int flag1, flag2, flag3, i, j, k, pos, max, pfcount = 0;
    printf("Enter number of frames: ");
    scanf("%d", &nf);

    printf("Enter number of pages: ");
    scanf("%d", &np);

    printf("Enter page reference string: ");

    for(i = 0; i < np; i++){
        scanf("%d", &page[i]);
    }

    for(i = 0; i < nf; i++){
        frame[i] = -1;    //setting all frame values -1 initially
    }

    //scanning all pages
    for(i = 0; i < np; i++){
        //setting both flag as 0
        flag1 = flag2 = 0;

        //scanning all frames
        for(j = 0; j < nf; j++){
            //checking page available in frame or not

```

```

        if(frame[j] == page[i]){
            //no need to replace simply skip
            flag1 = flag2 = 1;
            break;
        }
    }

    //checking flag1
    if(flag1 == 0){
        //scanning all frames
        for(j = 0; j < nf; j++){
            //checking frame empty or not
            if(frame[j] == -1){
                pfcount++;
                frame[j] = page[i];
                flag2 = 1;
                break;
            }
        }
    }

    //frame not empty, now we need to replace page
    if(flag2 == 0){
        //logic for replacing page
        flag3 = 0;

        for(j = 0; j < nf; j++){
            temp[j] = -1;

            for(k = i + 1; k < np; k++){
                if(frame[j] == page[k]){
                    temp[j] = k;
                    break;
                }
            }
        }

        for(j = 0; j < nf; j++){
            if(temp[j] == -1){
                pos = j;
                flag3 = 1;
                break;
            }
        }

        if(flag3 == 0){
            max = temp[0];
            pos = 0;

            for(j = 1; j < nf; j++){
                if(temp[j] > max){
                    max = temp[j];
                    pos = j;
                }
            }
        }
        frame[pos] = page[i];
        pfcount++;
    }

    printf("\n");
    //printing all frame values
    for(j = 0; j < nf; ++j){
        printf("%d\t", frame[j]);
    }
}

printf("\n\nTotal Page Faults = %d", pfcount);

```

```

    return 0;
}

```

Sample Input

```

Frames: 3
Pages: 12
Reference String:
1 2 3 4 1 2 5 1 2 3 4 5

```

Sample Output

```

Total Page Faults = 7

```

10. Priority Scheduling

Question:

Write a C program to implement Priority CPU Scheduling.

```

#include<stdio.h>
//Priority - Process Scheduling Algorithm

int main(){
    //declare variables
    int p[10],bt[10],pr[10],wt[10],tat[10],n,i,j,t;
    float awt=0,atat=0;

    //input
    printf("Enter no. of process (Max 10):\n");
    scanf("%d",&n);

    printf("Enter Burst Time and Priority of Each Process:\n");
    for(i=0;i<n;i++){
        p[i]=i+1; //inputting process number.
        printf("\nBurst Time of P[%d]:",i+1);
        scanf("%d",&bt[i]); //burst time input
        printf("Priority of P[%d]:",i+1);
        scanf("%d",&pr[i]); //inputting priority
    }

    //sorting and arranging process,priority and burst time
    //sorting on the basis of priority
    //sorting using selection sort..
    for(i=0;i<n;i++){
        for(j=i+1;j<n;j++){
            if(pr[i]>pr[j]){
                //swapping priority
                t=pr[i];
                pr[i]=pr[j];
            }
        }
    }

```

```

        pr[j]=t;

        //arranging burst time
        t=bt[i];
        bt[i]=bt[j];
        bt[j]=t;

        //arranging process
        t=p[i];
        p[i]=p[j];
        p[j]=t;
    }
}

//calculate waiting time
wt[0]=0; //wt of first process is always zero
for(i=1;i<n;i++){ //process
    wt[i]=0;
    for(j=0;j<i;j++){ //for getting burst time
        wt[i]=wt[i]+bt[j]; //wt calculated
    }

    awt=awt+wt[i]; //total wt calculated
}

//calculate turn around time
for(i=0;i<n;i++){
    tat[i]=wt[i]+bt[i]; //tat calculated

    atat=atat+tat[i]; //total tat calculated
}

//display
printf("\nProcess\tBT\tPR\tWT\tTAT\n");
for(i=0;i<n;i++)
    printf("P%d\t%d\t%d\t%d\t%d\n",p[i],bt[i],pr[i],wt[i],tat[i]);

printf("Average Waiting Time=%.2f ms\n",awt/n);
printf("Average Turn Around Time=%.2f ms\n",atat/n);

return 0;
}

```

Sample Input

Processes: 3

Burst Time	Priority
10	3
5	1
8	2

Sample Output

Process	BT	PR	WT	TAT
---------	----	----	----	-----

P2	5	1	0	5
P3	8	2	5	13
P1	10	3	13	23

Average Waiting Time = 6.00
Average Turnaround Time = 13.67

11. Producer-Consumer Problem

Question:

Write a C program to simulate the Producer-Consumer problem using semaphores.

```
#include<stdio.h>
#include<stdlib.h>

int mutex=1,full=0,empty=3,x=0;

int main()
{
    int n;
    void producer();
    void consumer();
    int wait(int);
    int signal(int);
    printf("\n1.Producer\n2.Consumer\n3.Exit");
    while(1)
    {
        printf("\nEnter your choice:");
        scanf("%d",&n);
        switch(n)
        {
            case 1:    if((mutex==1)&&(empty!=0))
                        producer();
                        else
                        printf("Buffer is full!!");
                        break;
            case 2:    if((mutex==1)&&(full!=0))
                        consumer();
                        else
                        printf("Buffer is empty!!");
                        break;
            case 3:    exit(0);
                        break;
        }
    }

    return 0;
}

int wait(int s)
{
    return (--s);
}

int signal(int s)
{
    return(++s);
}

void producer()
```

```

{
    mutex=wait(mutex);
    full=signal(full);
    empty=wait(empty);
    x++;
    printf("\nProducer produces the item %d",x);
    mutex=signal(mutex);
}

void consumer()
{
    mutex=wait(mutex);
    full=wait(full);
    empty=signal(empty);
    printf("\nConsumer consumes item %d",x);
    x--;
    mutex=signal(mutex);
}

```

Sample Input

```

1
1
2
3

```

Sample Output

```

Producer produces item 1
Producer produces item 2
Consumer consumes item 2
Program Exit

```

12. Readers-Writers Problem

Question:

Write a C program to solve the Readers-Writers problem using semaphores.

```

#include<semaphore.h>
#include<stdio.h>
#include<stdlib.h>
sem_t x,y;
pthread_t tid;
pthread_t writerthreads[100],readerthreads[100];
int readercount;

void *reader(void* param)
{
    sem_wait(&x);
    readercount++;
}

```

```

    if(readercount==1)
        sem_wait(&y);
        sem_post(&x);
        printf("\n%d reader is inside",readercount);
        sem_wait(&x);
        readercount--;
        if(readercount==0)
        {
            sem_post(&y);
        }
        sem_post(&x);
        printf("\n%d Reader is leaving",readercount+1);
    }

void *writer(void* param)
{
    printf("\nWriter is trying to enter");
    sem_wait(&y);
    printf("\nWriter has entered");
    sem_post(&y);
    printf("\nWriter is leaving");
}

int main()
{
    int n2,i;
    printf("Enter the number of readers:");
    scanf("%d",&n2);
    int n1[n2];
    sem_init(&x,0,1);
    sem_init(&y,0,1);
    for(i=0;i<n2;i++)
    {
        pthread_create(&writerthreads[i],NULL,reader,NULL);
        pthread_create(&readerthreads[i],NULL,writer,NULL);
    }
    for(i=0;i<n2;i++)
    {
        pthread_join(writerthreads[i],NULL);
        pthread_join(readerthreads[i],NULL);
    }
}

```

Sample Output

```

Reader 1 is inside
Reader 2 is inside
Writer is trying to enter
Reader 1 is leaving
Reader 2 is leaving
Writer has entered
Writer is leaving

```

13. SCAN Disk Scheduling

Question:

Write a C program to implement the SCAN Disk Scheduling algorithm.

```
#include<stdio.h>
#include<stdlib.h>

void printseq(int value){
    printf("%d->",value);
}

int main()
{
    int RQ[100],i,j,n,TotalHeadMoment=0,initial,size,move;
    printf("Enter the number of Requests\n");
    scanf("%d",&n);
    printf("Enter the Requests sequence\n");
    for(i=0;i<n;i++){
        scanf("%d",&RQ[i]);
    }
    printf("Enter initial head position\n");
    scanf("%d",&initial);
    printf("Enter total disk size\n");
    scanf("%d",&size);
    printf("Enter the head movement direction for high 1 and for low 0\n");
    scanf("%d",&move);

    // logic for Scan disk scheduling

    /*logic for sort the request array */
    for(i=0;i<n;i++){
        for(j=0;j<n-i-1;j++){
            if(RQ[j]>RQ[j+1]){
                int temp;
                temp=RQ[j];
                RQ[j]=RQ[j+1];
                RQ[j+1]=temp;
            }
        }
    }

    int index;
    for(i=0;i<n;i++){
        if(initial<RQ[i]){
            index=i;
            break;
        }
    }

    // if movement is towards high value
    if(move==1)
    {
        for(i=index;i<n;i++){
            TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
            initial=RQ[i];
            printseq(initial);
        }
    }
}
```

```

    }
    // last movement for max size
    TotalHeadMoment=TotalHeadMoment+abs(size-RQ[i-1]-1);
    initial = size-1;
    for(i=index-1;i>=0;i--)
    {
        TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
        initial=RQ[i];
        printseq(initial);
    }
}
// if movement is towards low value
else
{
    for(i=index-1;i>=0;i--)
    {
        TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
        initial=RQ[i];
        printseq(initial);
    }
    // last movement for min size
    TotalHeadMoment=TotalHeadMoment+abs(RQ[i+1]-0);
    initial =0;
    for(i=index;i<n;i++)
    {
        TotalHeadMoment=TotalHeadMoment+abs(RQ[i]-initial);
        initial=RQ[i];
        printseq(initial);
    }
}

printf("\nTotal head movement is %d",TotalHeadMoment);
return 0;
}

```

Sample Input

Requests:
 98 183 37 122 14 124 65 67
 Initial Head:
 53
 Disk Size:
 200
 Direction:
 1

Sample Output

65->67->98->122->124->183->37->14->
 Total head movement is 331

14. Semaphore Demonstration

Question:

Write a C program to demonstrate the use of semaphores for mutual exclusion.

```
// C program to demonstrate working of Semaphores
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t mutex;

void* thread(void* arg)
{
    //wait
    sem_wait(&mutex);
    printf("\nEntered..\n");

    //critical section
    sleep(4);

    //signal
    printf("\nJust Exiting...\n");
    sem_post(&mutex);
}

int main()
{
    sem_init(&mutex, 0, 1);
    pthread_t t1,t2;
    pthread_create(&t1,NULL,thread,NULL);
    sleep(2);
    pthread_create(&t2,NULL,thread,NULL);
    pthread_join(t1,NULL);
    pthread_join(t2,NULL);
    sem_destroy(&mutex);
    return 0;
}
```

Sample Output

Entered...

Just Exiting...

Entered...

Just Exiting...

15. SJF Scheduling

Question:

Write a C program to implement the Shortest Job First (SJF) Scheduling algorithm.

```
#include<stdio.h>
//Shortest Job First(SJF) - Process Scheduling Algorithm

int main() {
    //declare variables
    int p[10],bt[10],wt[10],tat[10],n,i,j,t;
    float awt=0,atat=0;

    //input
    printf("Enter no. of process\n");
    scanf("%d",&n);

    printf("Enter Burst Time\n");
    for(i=0;i<n;i++){
        p[i]=i+1; //inputting process number.
        scanf("%d",&bt[i]); //burst time input
    }

    //sorting and arranging process and burst time
    //sorting on the basis of burst time
    //sorting using selection sort..
    for(i=0;i<n;i++){
        for(j=i+1;j<n;j++){
            if(bt[i]>bt[j]){
                //swapping burst time
                t=bt[i];
                bt[i]=bt[j];
                bt[j]=t;

                //arrange process
                t=p[i];
                p[i]=p[j];
                p[j]=t;
            }
        }
    }

    //calculate wating time
    wt[0]=0; //wt of first process is always zero
    for(i=1;i<n;i++){ //process
        wt[i]=0;
        for(j=0;j<i;j++){ //for getting burst time
            wt[i]=wt[i]+bt[j]; //bt caluclated
        }

        awt=awt+wt[i]; //total wt calculated
    }

    //calculate turn around time
    for(i=0;i<n;i++){
        tat[i]=wt[i]+bt[i]; //tat caluclated

        atat=atat+tat[i]; //total tat calculated
    }

    //display
    printf("Process\tBT\tWT\tTAT\n");
    for(i=0;i<n;i++)
```

```

printf("P%d\t%d\t%d\t%d\n",p[i],bt[i],wt[i],tat[i]);

printf("Average Waiting Time=%.2f ms\n",awt/n);
printf("Average Turn Around Time=%.2f ms\n",atat/n);

return 0;
}

```

Sample Input

Processes: 4
 Burst Time:
 6
 8
 7
 3

Sample Output

Process	BT	WT	TAT
P4	3	0	3
P1	6	3	9
P3	7	9	16
P2	8	16	24

Average Waiting Time = 7.00
 Average Turnaround Time = 13.00

16. SSTF Disk Scheduling

Question:

Write a C program to implement the SSTF (Shortest Seek Time First) Disk Scheduling algorithm.

```

//Shortest Seek Time First (SSTF)
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int RQ[100],i,n,TotalHeadMoment=0,initial,count=0;
    printf("Enter the number of Requests\n");
    scanf("%d",&n);
    printf("Enter the Requests sequence\n");
    for(i=0;i<n;i++)
        scanf("%d",&RQ[i]);
    printf("Enter initial head position\n");
    scanf("%d",&initial);

    // logic for sstf disk scheduling

```

```

/* loop will execute until all process is completed*/
while(count!=n)
{
    int min=1000,distance,index;
    for(i=0;i<n;i++)
    {
        distance=abs(RQ[i]-initial);
        if(min>distance)
        {
            min=distance;
            index=i;
        }
    }
    TotalHeadMoment=TotalHeadMoment+min;
    initial=RQ[index];
    //printing movement
    printf("%d->",initial);
    // 1000 is for max
    // you can use any number
    RQ[index]=1000;
    count++;
}

printf("\nTotal head movement is %d",TotalHeadMoment);
return 0;
}

```

Sample Input

Requests:
 98 183 37 122 14 124 65 67
 Initial Head:
 53

Sample Output

65->67->37->14->98->122->124->183->
 Total head movement is 236

Operating System Viva Questions with Answers

1. What is an Operating System (OS)?

Answer: An Operating System is system software that manages computer hardware, software resources, and provides services to application programs.

2. What are the main functions of an Operating System?

Answer:

- Process management
- Memory management
- File management
- Device management
- Security
- User interface

3. What is a Process?

Answer: A process is a program that is currently executing.

4. What is a Thread?

Answer: A thread is the smallest unit of CPU execution inside a process.

5. What is CPU Scheduling?

Answer: CPU Scheduling is the process of selecting which process gets the CPU next.

6. What is FCFS Scheduling?

Answer: FCFS (First Come First Serve) executes processes in the order they arrive.

Advantage: Simple

Disadvantage: High waiting time

7. What is SJF Scheduling?

Answer: Shortest Job First executes the process with the smallest burst time first.

8. What is Priority Scheduling?

Answer: Processes are executed according to their priority.

9. What is Round Robin Scheduling?

Answer: Every process gets a fixed time quantum repeatedly.

10. What is Burst Time?

Answer: Time required by a process to execute.

11. What is Waiting Time?

Answer: Time a process waits in the ready queue before execution.

12. What is Turnaround Time?

Answer: Turnaround Time = Completion Time – Arrival Time.

13. What is Memory Management?

Answer: It is the management of main memory by allocating and deallocating memory to processes.

14. What is Virtual Memory?

Answer: Virtual memory allows execution of programs larger than physical memory using secondary storage.

15. What is Paging?

Answer: Paging divides memory into fixed-size pages and frames.

16. What is a Page Fault?

Answer: A page fault occurs when a required page is not found in main memory.

17. What is Page Replacement?

Answer: It is the process of replacing pages in memory when all frames are full.

18. What is FIFO Page Replacement?

Answer: The oldest page is replaced first.

19. What is LRU Page Replacement?

Answer: The least recently used page is replaced.

20. What is Optimal Page Replacement?

Answer: It replaces the page that will not be used for the longest time in the future.

21. Which page replacement algorithm gives minimum page faults?

Answer: Optimal Page Replacement.

22. What is Thrashing?

Answer: Excessive page faults causing poor system performance.

23. What is Deadlock?

Answer: Deadlock occurs when two or more processes wait indefinitely for each other.

24. What are the four necessary conditions for Deadlock?

Answer:

- Mutual Exclusion
- Hold and Wait
- No Preemption
- Circular Wait

25. What is Banker's Algorithm?

Answer: It is a deadlock avoidance algorithm that checks whether the system remains in a safe state.

26. What is Safe State?

Answer: A state where all processes can complete without deadlock.

27. What is Semaphore?

Answer: A semaphore is a synchronization tool used to control access to shared resources.

28. What is Mutex?

Answer: Mutex is a mutual exclusion lock allowing only one process/thread to access a resource at a time.

29. What is the Producer-Consumer Problem?

Answer: It is a synchronization problem where producers generate data and consumers use it.

30. What is the Readers-Writers Problem?

Answer: Multiple readers can read simultaneously, but writers require exclusive access.

31. What is File Management?

Answer: Managing creation, deletion, storage, and retrieval of files.

32. What is a File?

Answer: A file is a collection of related data stored on secondary storage.

33. What is Disk Scheduling?

Answer: Disk scheduling determines the order of servicing disk requests.

34. Name Disk Scheduling Algorithms.

Answer:

- FCFS
- SSTF
- SCAN
- C-SCAN
- LOOK
- C-LOOK

35. What is SSTF?

Answer: Shortest Seek Time First services the nearest disk request first.

36. What is SCAN Scheduling?

Answer: The disk arm moves in one direction servicing requests, then reverses direction.

37. What is C-SCAN?

Answer: The disk arm moves in one direction only and jumps back to the beginning after reaching the end.

38. What is LOOK Scheduling?

Answer: Similar to SCAN, but the head reverses direction at the last request instead of the disk end.

39. What is C-LOOK Scheduling?

Answer: Similar to C-SCAN, but the head jumps from the last request to the first request instead of the disk end.

40. Which disk scheduling algorithm gives the best average seek time?

Answer: SSTF usually provides the lowest average seek time, though it may cause starvation.